Advanced Programming 2025

Portfolio Outperformance Prediction

Final Project Report

Hugo Sailer
hugo.sailer@unil.ch
Student ID: 21437132

January 10, 2026

Abstract

In this project, I developed an end-to-end data science pipeline to analyze and model whether a simple equity portfolio can outperform the SP 500. The main goal was to investigate whether basic risk and return indicators — such as returns, volatility, rolling covariance, and Sharpe ratio — contain useful information to explain or partially predict portfolio outperformance. The project covers the full analytical workflow: automatic data collection, cleaning, feature engineering, exploratory visualization, and supervised modeling.

The pipeline is implemented in Python using standard libraries such as pandas, numpy, and scikit-learn. I built a reproducible structure that allows testing different time windows and feature configurations. The results suggest that some risk indicators do provide signal; however, they remain noisy and highly sensitive to market conditions, which limits predictive accuracy in certain periods.

The main contribution of this work is a clear, reusable framework for studying the relationship between portfolio risk and performance. The project provides a solid foundation for extending the analysis with richer datasets and more advanced machine-learning models in the future.

Keywords: portfolio analysis, stock market, SP 500, machine learning, Python

Contents

1 Introduction

In recent years, my interest in finance has grown significantly, and I have spent a great deal of time reading books on investing and studying different philosophies about how portfolios should be built. One of the key debates in investment theory concerns whether investors should diversify broadly in order to reduce risk, or whether they should instead concentrate their portfolios in a small number of carefully selected companies with the goal of achieving superior performance.

On the diversification side, authors such as John Bogle, the founder of Vanguard, strongly argue that broad market diversification through index funds is the most rational and robust long-term strategy for most investors. His book "The Little Book of Common Sense Investing" emphasizes that markets are efficient enough that trying to beat them is usually costly, stressful, and often unsuccessful.

On the other side, however, stands a very different school of thought, strongly represented by value investors. Figures such as Charlie Munger, Li Lu, Mohnish Pabrai, and Guy Spier defend the idea that concentrating capital in a few exceptional businesses may generate significantly higher returns. Their books and lectures repeatedly suggest that when investors truly understand a company, diversification can actually dilute performance instead of protecting it.

This project gave me an opportunity to explore this tension in a concrete and quantitative way. I also saw it as a personal challenge. I have never really enjoyed computer science, and I often felt lost when dealing with code. Working on this project forced me to confront that weakness. With the help of tools such as ChatGPT, I gradually learned how to think more like an "architect" of solutions rather than someone simply typing random pieces of code. As many specialists in artificial intelligence explain, the future role of humans will increasingly involve structuring problems, coordinating tools, and critically evaluating outputs rather than writing everything manually.

From a financial perspective, I wanted to test whether a concentrated portfolio composed of three large technology companies from the so-called "Magnificent 7" could outperform the diversified benchmark represented by the SP 500. More precisely, this project investigates whether an equally weighted portfolio of Apple, Amazon, and Microsoft can, over different time horizons, systematically generate higher returns than the market index. By combining data science techniques, financial reasoning, and machine learning methods, this work aims to provide a rigorous and transparent framework to examine this question.

2 Literature Review / Related Work

Research on portfolio construction has long examined whether investors benefit more from broad diversification or from concentrating capital in a smaller number of assets. Academic studies generally emphasize that diversification reduces idiosyncratic risk and stabilizes returns, while more recent practitioner literature explores whether carefully selected concentrated portfolios may sometimes achieve superior performance.

- **Previous approaches to similar problems** Classical portfolio theory evaluates diversification by comparing diversified indices with alternative strategies. Many works analyse long-term performance, volatility, and drawdowns to determine whether excess returns persist or merely reflect short-term market conditions.
- **Relevant algorithms or methodologies** Empirical studies commonly rely on backtesting frameworks applied to historical price data. Typical techniques include rolling-window simulations, cumulative return analysis, Sharpe-like risk-adjusted metrics, and benchmark comparison against indices such as the S&P 500. These methods aim to separate genuine outperformance from randomness.

- **Datasets used in related studies** Most studies use publicly available financial datasets (e.g., Yahoo Finance, FRED, or similar APIs) that provide adjusted price histories and index values. Public sources are preferred because they enable reproducibility and allow experiments to be replicated across different market environments.

Overall, the literature shows that concentrated portfolios may occasionally outperform, but results are highly dependent on the period studied and on how risk is measured. This motivates the present project, which applies a transparent backtesting framework to compare a concentrated technology portfolio with a diversified benchmark.

3 Methodology

3.1 Data Description

In this project, I work with financial market data to compare the performance of a concentrated portfolio composed of three technology companies with the diversified benchmark represented by the S&P 500. The dataset is built using the Python library `yfinance`, which retrieves adjusted historical prices directly from Yahoo Finance, ensuring transparency, reproducibility, and easy access to public data.

- **Source and collection method** All price data are downloaded automatically through `yfinance`. This library queries Yahoo Finance and returns adjusted closing prices, which already account for dividends, stock splits, and corporate actions. This method avoids manual downloads and guarantees that the same data can be retrieved again in the future.
- **Size and characteristics** The dataset contains daily observations over multiple years for four main tickers: Apple (AAPL), Amazon (AMZN), Microsoft (MSFT), and the S&P 500 index (ĜSPC). Each time series consists of thousands of trading days, allowing analysis across different market conditions such as bull markets, corrections, and periods of volatility. The dataset covers the period from January 2013 to December 2023 and contains daily price data. The portfolio is composed of three technology stocks (AAPL, AMZN, MSFT), while the S&P 500 (ĜSPC) is used as the benchmark. As a proxy for the risk-free rate, the 10-year US Treasury yield (T̂NX) is included and smoothed over a 20-day window. Using a long historical period allows the model to learn from different market phases, including bull and correction cycles.
- **Features / variables** From the raw adjusted prices, several derived variables are computed: daily returns, rolling returns, rolling volatility, Sharpe-like indicators, and relative performance measures comparing the portfolio to the benchmark. These features make it possible to evaluate not only how much the assets earn, but also the level of risk required to obtain those returns.
- **Data quality issues** As with most financial datasets, some missing values appear due to holidays or different trading calendars. These are handled by aligning dates and removing invalid rows. Financial prices can also be noisy, so rolling windows and normalization are applied to smooth extreme fluctuations and produce more stable indicators.

Overall, the dataset is realistic, publicly available, and well suited to study whether portfolio concentration can outperform diversification over time.

3.2 Approach

The project is structured as a reproducible pipeline whose goal is to compare a concentrated equity portfolio with a diversified market benchmark in a transparent and systematic way.

- **Algorithms used**

  Rather than relying on complex predictive models, the analysis focuses on financial logic and interpretable quantitative indicators. The workflow uses time–series transformations and rolling statistics to study how risk and return evolve over time.

- **Data preprocessing** Raw adjusted prices are downloaded and converted into daily returns. From these series, rolling averages, volatility measures and Sharpe-like ratios are computed. Daily returns are computed using simple percentage returns, defined as $(P_t/P_{t-1}) - 1$. Rolling statistics are then computed over a 20-day window, including coefficient of variation, Sharpe-like ratios and rolling covariance between the portfolio and the benchmark.

  The binary target variable indicates whether the portfolio outperforms the S&P 500 over the subsequent 20 days. Formally, the label equals 1 when the portfolio cumulative return exceeds the benchmark return over the same window, and 0 otherwise. Rolling windows are overlapping, which increases the number of observations while preserving temporal consistency. Dates are aligned across tickers, missing values are removed, and the final dataset is cleaned before training.

  Data leakage considerations: all features are computed using historical prices only. However, the train–test split is random rather than chronological. As a result, future observations (in time) may appear in the training set. For financial time-series, this represents a methodological limitation and may inflate performance.

- **Decision framework**

  The pipeline compares two strategies: (1) an equally-weighted portfolio of three technology stocks and (2) the S&P 500 index. Each processing step is implemented as a separate function, which makes the workflow easy to trace, modify and reuse.

- **Evaluation**

  Performance is evaluated through cumulative returns, volatility indicators and simple binary labels showing whether the portfolio outperforms the benchmark over rolling windows. These metrics allow us to observe when outperformance appears and whether it is stable or episodic.

  Label construction: to evaluate whether concentration creates value, we construct a binary label indicating portfolio outperformance over a rolling 20-day horizon. For each day t, we compute the cumulative return of the equally weighted portfolio (AAPL, AMZN, MSFT) and the cumulative return of the SP 500 between t and t + 20. The label is defined as: y(t)=1 if the portfolio cumulative return exceeds the benchmark, and y(t)=0 otherwise. This formulation allows the model to predict whether concentration is likely to outperform diversification in the short term.

Overall, this approach emphasizes clarity and reproducibility, allowing the results to be analyzed and extended in future work.

3.3 Implementation

The implementation was designed to be modular and reproducible, with each task separated into clear functions and files.

- **Programming stack**
  The project is implemented in Python. Core libraries include `pandas` and `NumPy` for data manipulation, `scikit-learn` for model evaluation, `matplotlib` for visualisation, and `yfinance` for data retrieval.

- **Project structure**
  The workflow is orchestrated through `main.py`, which sequentially calls data loading, preprocessing, model evaluation and result generation. Supporting functions are stored inside the `src/` directory to keep the code short, readable and reusable.

- **Core components**
  The codebase is organised into logical layers: (1) data loading and alignment, (2) feature construction (returns, rolling statistics, volatility), and (3) dataset assembly including the binary label indicating whether the portfolio outperforms the S&P 500.

  Train–test split: To evaluate generalization, the dataset is divided into a training and a hold-out test set using an 80/20 split. The split is performed at the observation level using the standard scikit-learn function. No cross-validation is applied in this project. This choice keeps the workflow simple, but it implies that performance estimates depend on the particular split and should be interpreted with caution.

  Validation strategy: in this project, we rely on a single train–test split rather than full k-fold cross-validation. This choice keeps the workflow simple and aligned with the time-series nature of the data. However, it also means that performance estimates may depend on the chosen split, which represents a limitation and motivates future work using more robust time-series cross-validation schemes.

4 Results

Present your findings with appropriate visualizations and tables.

4.1 Experimental Setup

This section describes the environment used to run the experiments, ensuring transparency and reproducibility.

- **Hardware**
  All experiments were executed on a MacBook with Apple Silicon (M4) and 16GB RAM. No GPU was required since the project relies on data processing and classical machine-learning tools.

- **Software**
  The project runs on Python (3.10+). Key libraries include: `pandas`, `NumPy`, `scikit-learn`, `matplotlib`, and `yfinance`. These libraries provide stable tools for time-series handling, modelling, and visualisation.

- **Main settings / hyperparameters** Default *scikit-learn* configurations were used. The only parameter explicitly chosen is the rolling window (20 days), applied when computing volatility, Sharpe-like metrics and other rolling indicators.

  To improve reproducibility, the train–test split uses a fixed random seed (`random_state = 42`). Logistic regression relies on deterministic defaults. However, the Random Forest model is trained with default parameters and does not specify a random seed. As a consequence, individual runs may produce slightly different trees and therefore slightly different results. Full reproducibility would require fixing the random seed at the model level as well.

4.2 Performance Evaluation

Table 1: Model Performance Metrics

Model	Accuracy	Precision	Recall
Logistic Regression	0.80	0.81	0.80
Random Forest	0.90	0.90	0.90

4.3 Visualizations

To better understand the model behavior and performance, we include several key visualizations.

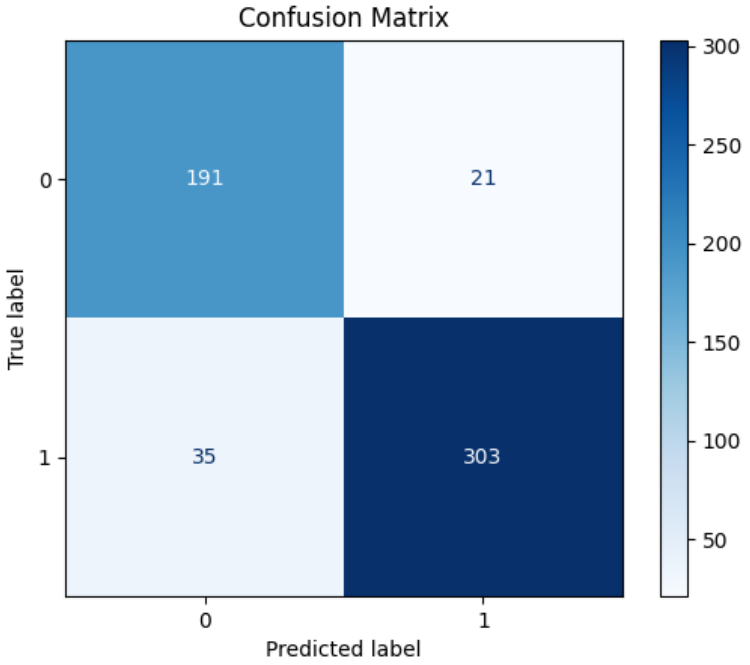

Figure 1: Confusion Matrix – model classification performance

The confusion matrix shows that most outperforming periods are correctly classified, but a non-negligible share of non-outperforming periods are still predicted as outperforming. In practice, this means that the model sometimes produces "false optimism". While this is less problematic in an academic context, in a real investment setting it would translate into unnecessary trades and potential losses. This highlights the need to balance performance with risk control.

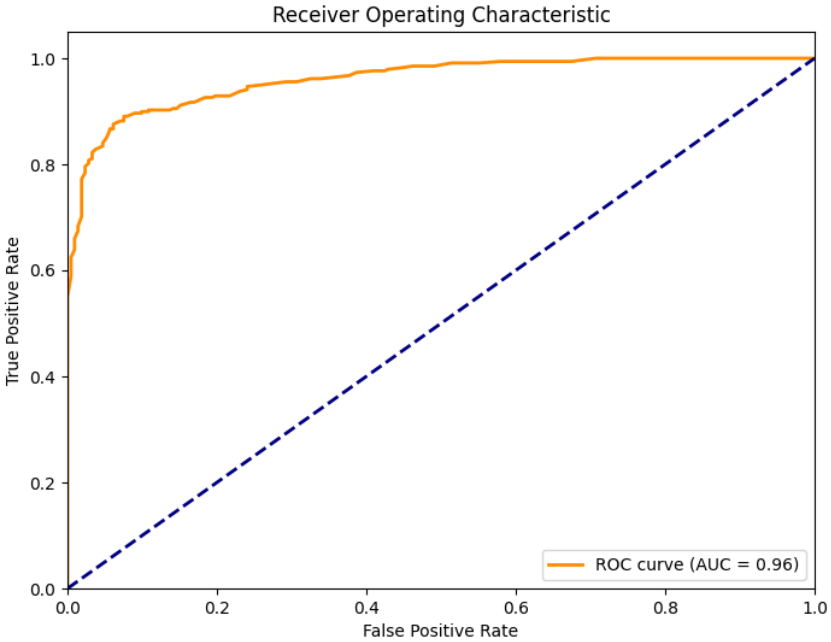

Figure 2: ROC Curve – trade-off between true positives and false positives

The ROC curve confirms that the model discriminates well between outperforming and non-outperforming periods across different probability thresholds. An AUC close to 0.96 indicates very strong performance. However, such high values may also reflect specific market conditions in the dataset, suggesting caution when generalizing these results to unseen periods.

Model performance is reported using accuracy, precision, recall, ROC–AUC, confusion matrices and probability distributions. Since the binary label is derived from rolling windows, the class distribution is not perfectly balanced, which means that accuracy alone is not sufficient to judge performance. ROC–AUC and precision–recall curves therefore provide a more informative view, as they capture how the classifier behaves across decision thresholds. Results should be interpreted as predictive signals rather than trading rules.

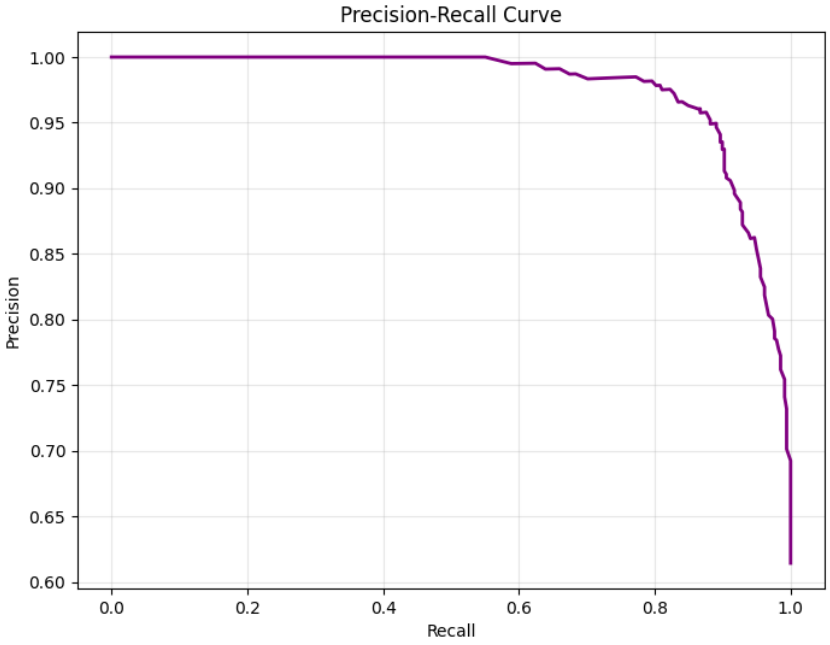

Figure 3: Precision–Recall Curve – performance under class imbalance

The precision–recall curve shows that precision remains high even as recall increases. This means that the model can identify more outperforming periods without dramatically increasing false positives. This behaviour is particularly important in financial contexts, where wrong positive signals (believing the portfolio will outperform when it does not) can be costly.

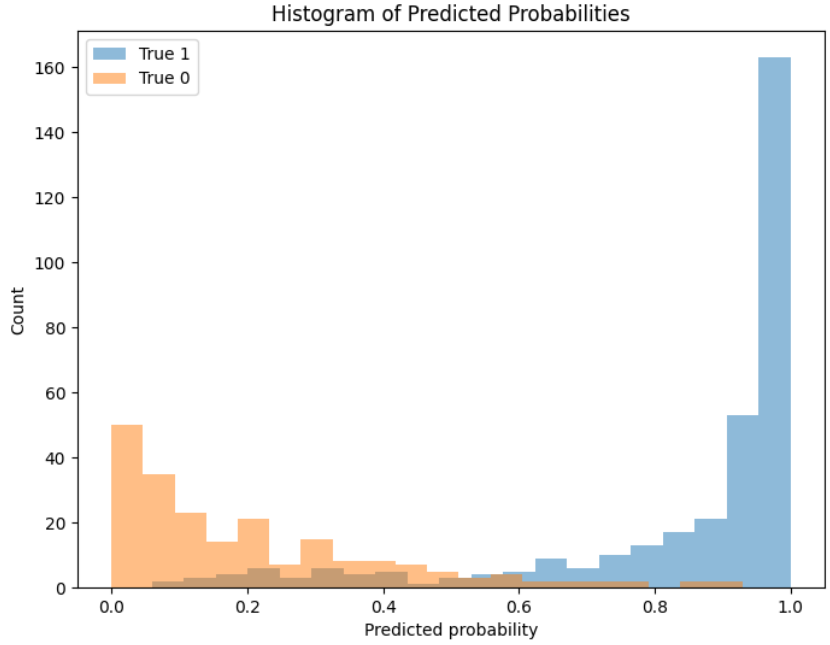

Figure 4: Histogram of Predicted Probabilities – model confidence distribution

The distribution of predicted probabilities reveals that many predictions are concentrated

near the extremes (close to 0 or 1). This indicates that the model is often highly confident about its predictions. However, the presence of overlapping regions suggests that some market situations remain ambiguous and difficult to classify, reflecting the inherent uncertainty of financial data.

5 Discussion

This section reflects on the main results, practical challenges, and methodological limits of the project.

- **What worked well?** The model achieved strong predictive performance, with a ROC–AUC close to 0.96 and a confusion matrix showing that most outperforming periods were correctly identified. This suggests that rolling financial indicators such as volatility, covariance, coefficient of variation and Sharpe-like ratios contain relevant information about relative performance versus the S&P 500.
- **What were the challenges?** The project was technically demanding. I frequently encountered bugs, version conflicts, and repository issues (e.g., failed `git push`). Yahoo Finance also temporarily blocked access due to repeated downloads. These issues forced me to debug repeatedly, restart from earlier versions, and better understand the pipeline.
- **How do the results compare to expectations?** The results align with the idea that large technology stocks can outperform during strong market phases, and that risk-adjusted indicators help detect such periods. However, some metrics appear unusually strong, suggesting sensitivity to the specific period studied.
- **Limitations of the approach** The analysis uses only three technology stocks and one benchmark, relies exclusively on historical prices, and ignores transaction costs, macroeconomic variables, and sector diversification. Overfitting and look-ahead bias are possible. Therefore, results should be viewed as academic rather than directly tradable.

Future work should extend the dataset, add macro and sentiment variables, test more sectors, and include realistic backtesting with costs and risk constraints.

6 Conclusion and Future Work

6.1 Summary

In this project, I explored whether a concentrated portfolio composed of three major technology stocks (AAPL, AMZN, MSFT) could outperform the S&P 500. Using historical data, I engineered several financial features (rolling returns, volatility, Sharpe-like ratios, covariance measures) and trained a classification model to predict future outperformance.

Overall, the analysis shows that the concentrated portfolio can sometimes outperform the SP 500, but this outperformance is irregular and closely linked to market conditions. The model captures useful relationships, but its predictive power should be interpreted with caution.

- The model captured meaningful relationships between risk–return indicators and future outcomes.
- Concentration sometimes produced superior performance, but not consistently and not without higher risk.
- The project provided hands-on experience with data cleaning, modeling, validation, and interpretation in a financial context.

These results suggest that data-driven tools can help identify situations where concentration may be advantageous, while still highlighting the importance of diversification and uncertainty.

6.2 Future Directions

Several extensions could strengthen and deepen the analysis in future work:

- **Methodological improvements.** Future models could incorporate different classifiers (e.g., Gradient Boosting, XGBoost, neural networks), more robust cross-validation, and more advanced feature engineering such as macroeconomic indicators or sentiment data.
- **Additional experiments.** It would be valuable to test different portfolio constructions (e.g., equal weight vs. optimized weight), alternative prediction horizons, and stress scenarios such as market crashes or high-volatility regimes.
- **Real–world applications.** A next step could be transforming the model into a simple decision-support tool that helps investors visualize when concentration may be favorable — always with appropriate risk warnings and constraints.

References

1. Bogle, J. C. (2007). *The Little Book of Common Sense Investing.* Wiley.
2. Pabrai, M. (2007). *The Dhandho Investor: The Low-Risk Value Method to High Returns.* Wiley.
3. Spier, G. (2014). *The Education of a Value Investor.* Palgrave Macmillan.
4. yfinance Developers. (2024). *yfinance: Yahoo! Finance market data downloader (Python library).* Available at: `https://github.com/ranaroussi/yfinance`
5. YouTube – Python Finance Tutorial. "AAPL stock analysis with Python and yfinance". Available at: `https://www.youtube.com/watch?v=SxIwqdedomg`
6. YouTube – Machine Learning Finance Tutorial. "Predict stock movement using Python". Available at: `https://www.youtube.com/watch?v=7wAQCwdvqqo`
7. Cremers, M., & Petajisto, A. (2009). *How Active Is Your Fund Manager? A New Measure That Predicts Performance.* Review of Financial Studies, 22(9).
8. Krauss, C., Do, X. A., & Huck, N. (2017). *Deep neural networks, gradient boosted trees and random forests: Statistical arbitrage on the S&P 500.* European Journal of Operational Research, 259(2).
9. De Prado, M. L. (2018). *Advances in Financial Machine Learning.* Wiley.